COMSATS UNIVERSITY ISLAMABAD

(ABBOTTABAD CAMPUS)

ASSIGNMENT- 3

MAIN TOPIC: **AI TESTING TOOLS**

COURSE: SOFTWARE QUALITY ENGINEERING

COURSE TEACHER: SIR. MUKHTIAR ZAMIN

SUBMITTED BY:

MUHAMMAD Moazam **CIIT/FA23-BSE-071/ATD**

CLASS: **BSE-6B**

SUBMITTED ON: 17TH JULY, 2026

Table of contents

QUESTION: **3**

SOLUTION: **3**

DESCRIPTION: 3
STEP-1: SETTING VIRTUAL ENVIRONMENT: 3
STEP-2: CODE: 3
STEP-3: CREATING A NEW COLLECTION IN POSTMAN: 7
STEP-4: GET API TEST: 7
STEP-5: GET API TEST: 8
STEP-6: GET API TEST: 8
STEP-7: POST API TEST: 9
STEP-8: POST API TEST: 9
STEP-9: PUT API TEST: 10
STEP-10: PUT API TEST: 10
STEP-11: DELETE API TEST: 11

QUESTION:

Summarize the Use, Advantages and Productivity of Code Generation with Playwright.

SOLUTION:

Introduction

Software testing is one of the most important phases of the Software Development Life Cycle (SDLC). Traditionally, testers manually write test scripts to verify that an application behaves correctly. This process is often time-consuming, repetitive, and prone to human error. With the advancement of Artificial Intelligence (AI), developers and testers can now use AI-powered code generation tools to automatically create test scripts, significantly improving productivity and reducing development time.

One of the most widely used frameworks for browser automation and end-to-end testing is **Playwright**, developed by Microsoft. When combined with AI-powered code generation tools such as Google Gemini, ChatGPT, GitHub Copilot, or Antigravity, Playwright enables rapid creation, execution, and maintenance of automated tests.

What is Code Generation?

Code Generation (CodeGen) is the process of automatically generating source code using AI or software tools instead of writing the code manually. AI models understand natural language instructions and generate executable code based on user requirements.

For example, instead of writing a login test manually, a tester can simply provide the following prompt:

> **"Generate a Playwright test case for the login page with valid and invalid credentials."**

The AI generates the required Playwright code automatically.

What is Playwright?

Playwright is an open-source browser automation framework developed by Microsoft for end-to-end testing of modern web applications. It supports multiple programming languages including JavaScript, TypeScript, Python, Java, and C#.

Playwright supports testing across all major browsers:

- Chromium (Google Chrome, Microsoft Edge)

- Firefox
- WebKit (Safari)

Because Playwright uses auto-waiting mechanisms and reliable element handling, it produces more stable automated tests compared to many older automation frameworks.

Code Generation with Playwright

When AI is integrated with Playwright, testers no longer need to write every line of automation code manually. Instead, they describe the required test in natural language, and the AI generates the Playwright test script.

For example, the prompt:

> "Generate a Playwright script that opens Google, searches for 'Software Quality Engineering', and verifies the search results page."

The AI generates a complete Playwright automation script within seconds.

This significantly reduces the effort required for writing repetitive test cases.

Advantages of Code Generation with Playwright

1. Faster Test Development

AI generates automation scripts within seconds, reducing the time spent writing repetitive code.

2. Reduced Human Errors

Automatically generated scripts reduce syntax mistakes and common programming errors.

3. Increased Productivity

Test engineers spend less time coding and more time designing meaningful test scenarios.

4. Easier Maintenance

AI can regenerate or update scripts whenever the application's user interface changes.

5. Cross-Browser Testing

Playwright automatically supports Chromium, Firefox, and WebKit, allowing one test to run on multiple browsers.

6. Improved Test Coverage

AI can quickly generate multiple positive and negative test cases that might otherwise be overlooked.

7. Beginner Friendly

Even users with limited programming experience can create automated tests using natural language prompts.

Productivity Improvements

Traditional Testing	AI + Playwright
Manual coding	AI-generated code
Hours to write tests	Minutes to generate tests
More human errors	Reduced errors
Difficult maintenance	AI-assisted maintenance
Limited test coverage	Broader test coverage

QUESTION:

What is the Use of MCP (Model Context Protocol) in Software Quality Engineering (SQE)?

SOLUTION:

Introduction

Modern AI assistants are becoming increasingly capable of generating code, explaining bugs, and automating software testing. However, AI alone cannot directly interact with external tools such as GitHub, databases, testing frameworks, browsers, or issue tracking systems.

To solve this problem, **Anthropic introduced the Model Context Protocol (MCP)**, an open standard that enables AI models to securely communicate with external applications and tools through a common interface.

MCP acts as a bridge between AI assistants and software engineering tools.

What is Model Context Protocol (MCP)?

Model Context Protocol (MCP) is an open communication protocol that allows AI models to connect with external services, software tools, APIs, databases, and development environments.

Instead of giving AI only text-based instructions, MCP allows AI to access additional context and perform actions using connected tools.

It works similarly to how USB provides a universal standard for connecting different hardware devices.

Components of MCP

An MCP-based system generally consists of three components:

1. MCP Client

The AI assistant or application requesting information or actions.

Examples:

- ChatGPT
- Claude
- Gemini
- Antigravity

2. MCP Server

Provides access to external resources such as:

- GitHub
- Playwright
- Databases
- File systems
- APIs
- Testing tools

3. External Resources

The actual software systems where operations are performed.

Examples include:

- Source code repositories
- Bug tracking systems
- Continuous Integration servers
- Testing frameworks
- Cloud services

How MCP Works in SQE

A QA engineer asks the AI:

> "Run all login test cases and generate a defect report."

The AI uses MCP to:

1. Access the Playwright project.
2. Execute automated tests.
3. Collect execution logs.
4. Identify failed test cases.
5. Analyze failure reasons.
6. Generate a defect report.
7. Create a Jira issue automatically.

This entire workflow can be completed with minimal human intervention.

Applications of MCP in Software Quality Engineering

MCP enables AI to perform many QA-related activities, including:

- Automated test execution
- Bug analysis
- Test report generation

- Regression testing
- API testing
- Continuous Integration support
- Database validation
- Log analysis
- Requirement verification
- Test case generation

Benefits of MCP in SQE

- Reduces manual testing effort.
- Improves communication between AI and testing tools.
- Enables intelligent automation.
- Speeds up debugging.
- Supports continuous testing.
- Improves software quality.
- Reduces testing time and cost.

QUESTION:

What is Antigravity (antigravity.google) and How Can It Be Utilized in the SQE Domain for QA and Software Testing?

SOLUTION:

Introduction

Antigravity is Google's AI-powered software engineering platform designed to assist developers throughout the software development lifecycle. It combines the capabilities of Gemini AI with intelligent coding agents, automation tools, and development workflows to improve productivity.

Unlike traditional AI chatbots that only generate text, Antigravity is designed to perform software engineering tasks such as code generation, debugging, testing, documentation, and project analysis.

Features of Antigravity

Some of the key features include:

- AI-assisted coding
- Intelligent code completion
- Automated debugging
- Code explanation
- Playwright test generation
- Software documentation generation
- Project analysis
- Multi-file editing
- AI agents
- Integration with development tools
- MCP support

Using Antigravity for Software Quality Engineering

1. Test Case Generation

QA engineers can describe a software feature, and Antigravity automatically generates comprehensive test cases.

Example prompt:

> "Generate functional and boundary test cases for a login page."

2. Playwright Test Generation

Instead of writing automation scripts manually, Antigravity generates complete Playwright tests.

Example prompt:

> "Generate Playwright automation for user registration."

3. Bug Detection

Developers can paste source code into Antigravity and request bug analysis.

Example prompt:

> "Find logical errors in this authentication module."

4. Code Review

Antigravity reviews source code and suggests improvements for readability, security, and performance.

5. Test Script Optimization

Existing Playwright scripts can be optimized to improve reliability and maintainability.

6. Documentation Generation

It automatically creates documentation for APIs, functions, classes, and projects.

7. Regression Testing Support

AI assists testers by generating additional regression test cases whenever new features are introduced.

8. Debugging Assistance

Antigravity analyzes error logs and suggests possible fixes.

Example Workflow

A QA engineer receives a new login module.

The engineer asks Antigravity:

> "Generate Playwright tests for the login functionality."

Antigravity produces:

- Valid login test
- Invalid password test
- Invalid username test
- Empty username validation

- Empty password validation
- SQL injection attempt
- Password length validation
- Session timeout test

The tester reviews the generated code, executes the tests using Playwright, and verifies the application's behavior.

Benefits for QA Teams

- Faster automation development
- Reduced manual effort
- Better software quality
- Increased testing speed
- Higher productivity
- Improved collaboration between developers and testers
- AI-assisted debugging
- Easier maintenance of automation scripts

Conclusion

Artificial Intelligence is transforming Software Quality Engineering by automating repetitive and time-consuming testing tasks. AI-powered code generation with Playwright enables testers to create reliable automation scripts quickly, improving productivity and reducing manual effort. The Model Context Protocol (MCP) extends AI capabilities by securely connecting AI assistants with external tools such as Playwright, GitHub, databases, and issue trackers, enabling intelligent end-to-end testing workflows. Google's Antigravity further enhances software quality engineering by combining AI-assisted coding, debugging, code review, and automated test generation in a unified development environment.

Together, these technologies help organizations build higher-quality software faster, reduce testing costs, improve collaboration between developers and testers, and support modern continuous integration and continuous delivery (CI/CD) practices. As AI continues to evolve, its role in software testing and quality assurance will become increasingly significant, making these tools valuable assets for future software engineers and QA professionals.